Object Oriented Programming using C++
Classes and Objects

Y. T. Ching
Department of Computer Science
NYCU

OOP technology implements the important concept

- Encapsulation
- Inheritance
- Polymorphism

Encapsulation

- Encapsulation: the concept of binding together data and functions into one capsule or object.
- The concept is also known as data abstraction.
- The implementation enables data hiding: data in the object is hidden from the rest of the world. The data can be accessed through the interface functions.

Inheritance

- Inheritance: one of the most important tools in the implementation of code reusability.
- When using inheritance, new code can be derived or inherited from existing code.

Polymorphism

- Concept of using the same functions on different types of objects.
- Similar to function overload, but useful when the cases are "dynamic".

- class: abstract characterization of a set of objects.
  - example
    - a set of objects that consist of figures to draw.
    - figure class: the class all these objects belong.
  - class defines variables (data) and methods (operations) common to a set of objects.
  - class: prototype or generator of a set of objects.

- examples for abstract data type
  - a circle class:
    - Data, the representation of a circle, center and radius
    - methods: draw, move, erase,...
    - manipulates a circle by the methods without knowing the underlying data structure for the circle.
  - a priority queue
    - implementation of priority queue: array, list, heap,..
    - methods: delete min, insert arbitrary, ...
    - manipulates a priority queue by the methods.

- To access any member variable, use the membership operator (dot . operator).

```
speaker.impedance = 8;
amplifier.quantity = 1;
cout << amplifier.description;
```

- An example for C++ class

```
class string {
  char val[80];
public:
  void store (char *);
  int length();
};
```

- `string` class consists of data `val` and method `store (char *)` and `length()`.
- `string s, t`
- `s` and `t` are "instances" of `string`

- To access a member (data or method), C++ follows C's syntax
  - `s` an object, `x` a data member of `s`, `f` a method of `s`.
  - `s.x;, s.f(arguments);`
  - `ptr` a pointer to `s`
  - `ptr -> x;`
  - `ptr -> f(arguments);`

- Inheritance
  - produces a new class by deriving the new class from an already existing class.
  - derived class, inherited from the original class
  - in C++
    - derived class, subclass
    - base class, superclass
  - a derived class inherits all the data and methods from the existing class.
  - a derived class may add additional data or methods.
  - a derived class can redefine inherited method.
  - reusability

- An example

```
class pen {
  int x;
  int y;
  int status;
public:
  void set_status(int);
  void set_location(int,int);
};
```

- `pen p`, `p` an object of `pen`.
- `p.set_location(x,y);` a message, locate `p` at `(x,y)`.
- `p.set_status(1);` a message, set pen `p` on.

- hardware upgrade, we have a color pen. rewrite "colored_pen" or derive from "pen".

```
class colored_pen :  public pen {
  int color;
public:
  void set_color(int);
};
```

- colored_pen has all that pen (the base class) has, x,y,status,set_status,set_location
- colored_pen has some more, color, set_color.

- class hierarchy: A collection of classes obtained through inheritance.
  - container
    - set
    - sorted_set
    - list
    - stack
    - queue
- container is the base class of set and list
- sorted_set is derived from set
- stack and queue are derived from list

Polymorphism

- poly- many, morphism- forms
- polymorphism in OO programming: identically-named method that have different behavior depending on the type of object reference.
- Example
- a class `shape` and two subclasses `circle` and `line`
- `shape` has a method `draw`, and redefined in both subclasses.

- `shape* composite_fig[100];`
- In C++, we can assign a pointer to a base class object pointing to a derived class object.
- If we have a program to take user inputs, then draw many circles and lines.
- We don't know what `composite_fig[i]` has because it depends on user input.
- What polymorphism helps: if the i'th pointer points a `line` (`circle`), `line`'s (`circle`'s) `draw` will be invoked.
- `for (i=0;i<100;i++) composite_fig[i]->draw();`

Class

- Class: the foundation of OOP, very similar to a struct.

```
class class_name { // Name starts with upper case
private:
  // private variable and functions
public:
  // public variable and function
} object_list; // optional
```

- Class members: functions and variables declared inside the class.
- Member variables, also called data members.
- Member functions.
- C++ uses classes to create objects. An object is an instance of a class.
- class data members can be viewed as the object's attributes or properties.
- member functions are its behavior or methods.
- Spacecraft is an object with attributes such as velocity, acceleration, and position. And functions that manipulate the spacecraft's attributes describe its behavior.

- Accessing class member
- member can be data or function
- private: member can be accessed by the members of the same class.
- public: classes member can be accessed anywhere.
- protected: needed in inheritance.
- Data members are usually defined as private members.
- Functions (methods) are usually defined as public, serve as "interface",
- outside world uses methods to access or manipulate private data.
- In C++, the default access specifier is private.

```
class Jet {
  float acc, vel, dis;
  float getTime() {
    return vel/acc;
  }
public:
  void setValue(float x, float y) {
    acc=x; vel=y;
  }
  float getDisplacement() {
    return (vel*getTime())/2;
  }
};
```

```
int main()
{
  Jet plane;
  plane.setValue(40,65);
  cout << "The time during which the plane ";
  cout << "is being accelarated = ";
  cout <<plane.getTime(); // ERROR;
  cout << "\n The plane's displacement = ";
  cout << plane.getDisplacement();
  return 0;
}
```

Prog4_3

```
#include <iostream>
#include <cmath>
using namespace std;
class Resister {
private:
  int band1, band2, band3, band4;
  int resistance;
public:
  void getValue();
  void calResistance(){
  resistance=(((band1*10+band2)*pow(10,band3));
  }
  void printResistance();
};
```

```
void Resistor::getValues()
{
  cout<<"\t\tCOLORS:";
  cout<<"\n\n 0==BLACK ...."<<endl;
...........
  cout<<"10==GOLD 11==SILVER .." << endl;
  cout <<"Please select first band (1-9)==> ";
  cin >> band1;
  // then input all the other bands
  // band2, band3, band4
}
```

```
void Resistor::printResistance()
{
  char *tolerance;
  calResistance();
  if (band==10) tolerane = "+/- 5%";
  if (band==11) tolerane = "+/- 10%";
  if (band==12) tolerane = "+/- 20%";
  cout<<"\n\n The resistance is "
    <<resistance<<" ohms";
  cout<<" with "<< tolerance<<"
    tolerance." <<endl;
}
```

```
int main()
{
  Resistor res;
  res.getValues();
  res.printResistance();
  return 0;
}
```

Complex Example

```
#include <stdio.h>
/* structure definition */
typedef struct
{
  double real, imag;
} complex;

/* prototype */
int main(void);
complex cplx_set(double r, double i);
void cplx_print (complex c);
complex cplx_add (complex c1, complex c2);
complex cplx_sub (complex c1, complex c2);
```

```
int main (void)
{
  complex a, b, c, d;
  a = cplx_set(1.0,1.0);
  b = cplx_set(2.0,2.0);
  c = cplx_add (a,b);
  d = cplx_add (b, cplx_sub (c,a));
  printf ("c = ");
  cplx_print (d);
  printf ("and ");
  printf ("d = ");
  cplx_print (d);
  printf ("\n");
}
```

```
complex cplx_set(double r, double i);
{
  complex temp;
  temp.real = r;
  temp.imag = i;
  return temp;
}

void cplx_print (complex c)
{
  printf ("(%g, %g)", c.real, c.imag);
}
```

```
complex cplx_add (complex c1, complex c2)
{
  complex temp;
  temp.real = c1.real + c2.real;
  temp.imag = c1.imag + c2.imag;
  return temp;
}
complex cplx_sub (complex c1, complex c2)
{
  complex temp;
  temp.real = c1.real - c2.real;
  temp.imag = c1.imag - c2.imag;
  return temp;
}
```

Now modify the program in C++ syntax

```
#include <stdio.h>
/* structure definition */
typedef struct
{
  double real, imag;
} complex;
/* prototype */
int main(void);
complex cplx_set(double r, double i);
void cplx_print (complex c);
complex operator + (complex c1, complex c2);
complex operator - (complex c1, complex c2);
```

```
/* main program */
int main (void)
{
  complex a, b, c, d;
  a = cplx_set(1.0,1.0);
  b = cplx_set(2.0,2.0);
  c = a + b;
  d = b + c - a;
  printf ("c = ");
  cplx_print (d);
  printf ("and ");
  printf ("d = ");
  cplx_print (d);
  printf ("\n");
}
```

```
complex operator + (complex c1, complex c2)
{
  complex temp;
  temp.real = c1.real + c2.real;
  temp.imag = c1.imag + c2.imag;
  return temp;
}

complex operator - (complex c1, complex c2)
{
  complex temp;
  temp.real = c1.real - c2.real;
  temp.imag = c1.imag - c2.imag;
  return temp;
}
```

Now the C++ Code

```
#include <stdio.h>
class complex
{
  private:
    double real, imag;
  public:
    void set (double r, double i=0);
    void print();
    complex operator + (complex c);
    complex operator - (complex c);
}
int main (void); // prototype
```

```
int main(void)
{
  complex a, b, c, d;
  a.set(1.0,1.0);
  b.set(2.0,2.0);
  c = a + b;
  d = b + c - a;
  printf ("c = ");
  c.print ();
  printf ("and ");
  printf ("d = ");
  d.print ();
  printf ("\n");
}
```

```
void complex::set(r,i)
{
  real = r;
  imag = i;
}

void complex::print()
{
  printf ("(%g,%g)", real, imag);
}
```

```
complex complex::operator + (complex c)
{
  complex temp;
  temp.real = real + c.real;
  temp.imag = imag + c.imag;
  return temp;
}
complex complex::operator - (complex c)
{
  complex temp;
  temp.real = real - c.real;
  temp.imag = imag - c.imag;
  return temp;
}
```

A Stack Class

```
const int MaxStack = 10000;
const char EmptyFlag = '\0';
class Stack {
  enum {FullStack=MaxStack, EmptyStack=-1};
  enum {False=0, True=1};
  char items [MaxStack];
  int top;
```

```
public:
  // methods
  void init();        // set top
  void push(char);    // add an item
  char pop();         // remove an item
  int empty();        // no elements?
  int full();         // too many elements?
  void dump_stack();  // top to bottom
};
```

```
void Stack::init()
{
  top=EmptyStack;
}

void Stack::push(char c)
{
  if (full()) return;
  item[++top]=c;
}
```

```
char Stack::pop()
{
  if (empty()) return EmptyFlag;
  else return item[top--];
}

int Stack::full()
{
  if (top+1==FullStack) {
    cerr << "Stack full at "
      << FullStack << endl;
    return True;
  }
  else return False;
}
```

```
int Stack::empty()
{
  if (top==EmptyStack) {
  cerr << "Stack empty" << endl;
  return True;
  }
  else return False;
}

void stack::dump_stack()
{
  for (int i=top; i>=0; i--)
    cout << item[i]<<endl;
}
```

```
a code slice
Stack s1;
s1.init();
s1.push(‘a’);
s1.push(‘b’);
cout<<s1.pop();
s1.push(‘c’);
cout<<s1.pop();
s1.push(‘b’);
s1.push(‘c’);
s1.dump_stack();
```

- The `stack` class
- declaring a class does not allocate space.
- needs a method to initialize a newly created object (`init` in `stack` class).
- methods (`push`, `pop`) are `public`, can be accessed by code outside.
- data, `items[]` and `top` are private, cannot be accessed by code outside, but can be accessed by its own methods.
- manipulates the private data through the public method.

```
void f()
{
  Stack s;
  s.init;
  s.push(‘a’); // OK
  cout << s.item[0] << endl; // an error.
}
```

- The `Stack` exampls does not have a constructor.
- constructor and destructor are two special types of methods.
- invoke `init` after `s1` is created. If not, the `top` of `s1` contains junk.
- A reasonable approach: have a constructor that does the initialization automatically.

```
class Stack {
  enum {FullStack=MaxStack, EmptyStack=-1};
  enum {False=0, True=1};
  char items[MaxStack];
  int top;
public:
  Stack(); // the constructor
  void push (char); // add an item
  char pop (); // remove an item
  int empty(); // no elements?
  int full(); // too many elements?
  void dump_stack(); top to bottom
};

Stack::Stack()
{ top = EmptyStack; }

Stack s1; //create s1, constructor invoked
```

- Initializing variables before they are used is a common programming practice.
- Constructor: Can serve as a function to do initialization.
- A constructor
- In C++, a class must have at least a constructor and may have more than one constructor.
- constructor can be user-defined or compiler-generated.
- It is never explicitly called. A constructor function is automatically called whenever an object is declared.
- It has the same name as the class, no return type, not even `void`
- It can have arguments, including default arguments.
- Constructor should be `public`, so they can be called outside the class.
- Constructor cannot be inherited, each class should have it own constructors.

- We cannot initialize class data members within the class.

```
class Battery {
  float voltage=12.0;
  float capacity;
}
```

- It should be initialized in the constructor

```
Battery ()
{
  voltage = 12.0;
  capacity = 30.0;
}
```

- If there is a line `Battery bt;`
- `Battery ()` is invoked.
- we have
- `bt.voltage == 12`, and
- `bt.capacity == 30`
- What if we want values other than 12 and 30?
- We need another constructor (we can have more than one constructor).

```
Battery (float v, float c)
{
  voltage = v;
  capacity=c;
}
Battery bt(12.0, 30.0);
```

Or

```
Battery (float v=12.0, float c=30.0)
{
  voltage = v;
  capacity=c;
}
Battery bt1; or Battery b2(5.0, 20.0);
```

A class may have more than one constructor

```
class Color {
  float red;
  float green;
  float blue;
public:
  Color () {red=green=blue=0.0;}
    // 1st constructor
  Color (float, float, float); //2nd constructor
};

Color::Color(float r, float g, float b)
{ red=r; green=g; blue=b; }

Color c1; //invoke 1st constructor
Color c2(1.0,0.5,0.0); //invoke 2nd constructor
```

- parameterized constructor has initial values
- `Color c2(1.0);`
- `c2.red = 1.0` given by the argument.
- `c2.green=0.5`, and `c2.blue=0.0` are as initial values.
- initial values are given from left to right.
- `// The following is an error`
- `Color::Color(float r =0.5 , float g, float b=0.0) { red = r; green = g; blue = b;}`

- If there is no constructor, the compiler will generate a default one (Color()).
- If there is a constructor with arguments (not a default constructor). The compiler will not generate a default compiler,
- The user needs to provide arguments as specified when constructing an object.

```
class C1 {
  int iC1;
  float fC1;
public:
  // initialization in constructor body
  C1 (int a1, float a2) {iC1=a1; fC1=a2;}
};

// initialization in constructor header
C1 (int a1, float a2) :  iC1(a1), fC1(a2) {}
// initialization is done in the header, so
// empty constructor body

// mixed initialization
C1 (int a1, float a2) :  iC1 (a1) fC3=a2;
```

Header initialization is required if a class has a data member 1. const 2. object in some other class.

- Examples need header initialization

```
class C {
  const int x; // a const data member
  float y;
public:
  C(int a1, float a2) :  x(a1), y(a2) {}
} // x need header initialization, y need not.
class X {
  char moniker[20];
public:
  X(char* m){strcpy (moniker, m);}
};
class Y{
  int num;
  X x;
public:
  Y(int a1, char* a2):num(a1),x(a2) {}
};
```

- Copy constructor
- default constructor, no argument.
- constructors other than the default constructors must have an argument(s).
- The type of the argument cannot be the same as the class. A class C constructor must not take an argument of type C.
- Type of the argument can be a reference to the class- a copy constructor.
- constructs a new object by copying an existing one.
- compiler gives one if the programmer didn't provide one.

```
class Color{
  float red; float green; float blue;
public:
  Color() {red=green=blue=0.0;}
  Color(Color); // an error
};

class Color{
  float red; float green; float blue;
public:
  Color() {red=green=blue=0.0;}
  Color (float, float, float);
  Color (Color &); // Copy
};

Color::Color(Color& c)
{red=c.red; green=c.green; blue=c.blue;}
```

- Convert Constructor
- another one-parameter constructor (other than `C&`).
- Convert type `t` to type `C`.

```
class Clock {
  int hour;
  int min;
  int ap;
public:
  Clock() hour=12; min=0; ap=0;
  Clock (int);
  void print_time;
};

Clock::Clock (int time)
{
  min = time % 100;
  hour = time / 100;
  if (hour>12) {
  hour -= 12;
  ap = 1;
  }
  else ap=0;
```

```
void Clock::print_time()
{
  cout << setfill('0')<<setw(2) << hour
  << ':'  << setw(2) << min
  << setfill (' ');
  if (ap) cout << " PM";
  else cout << " AM";
  cout << endl;
}
// print out xx:xx XX
// 12:00 AM

// Clock::Clock(int time) convert time
// to Clock.
```

- if `c` is a `Clock` object,
- `c = Clock (1308); c.print_time(); // get 01:08 PM`
- `c = (Clock) 123;` a cast, 123 is convert to a `Clock` object by convert constructor, then copy to `c`.
- `c = 1155;` implicit type conversion same as the above, the compiler uses the convert constructor convert 1155 to a Colck then copy to c.

- Two tasks of the constructor
- initialization,
- dynamically allocates storage and then initializes the storage.

```
class C {
  char* text;
  ...
public:
  C();
};

C:C()
{
  text = new char[20]; // allocate
  strcpy (text, "North By Northwest");
}

C c; // create a C object
// allocate space char [20]
// and initialization
```

- destructor:
- a special method like a constructor that the programmer writes but does not call.
- automatically invoked whenever an object is destroyed.
- free memory allocated in the constructor.
- No argument, no return value.
- `Color::~Color()`

a piece of C code

```
typedef struct big_string {
  char* string;
  int size;
} BigStr;

#define OneBillion (1000000000)
void big_grabage()
{
  BigStr bs;
  bs.size = OneBillion;
  bs.string =
    (char*) calloc(bs.size,sizeof(char));
  /* process bs.string but forget to free it */
  ...
}
```

big garbage after execution the function

a C++ code

```
const long OneBillion = 1000000000;
class BigStr {
  char* string;
  long size;
public:
  BigStr();
  ~BigStr() {delete[] string;}
};

BigStr::BigStr()
{
  string = new char [OneBillion +1];
  for (long i=0;i<OneBillion;i++)
  string[i] = '$';
  string[OneBillion] = '\0';
  size = OneBillion;
}
```

A Zipcode class

- a class to manipulate zip codes of different lengths (5- or 9-digit codes).
- constructors allow zip codes to be generated from either character or integer.

```
#include <stdio.h>
#include <iostream.h>
#include <string.h>

const int MinZip = 5; //60607
const int BigZip = 10;
const int MaxZip = 32;
const int InitChar = '?';
const int Hyphen = '-';
const int SuffixLen = 4; // 60607-1234
```

```
class ZipC {
  char* code;
public:
  ZipC();
  ZipC(const char*);
  ZipC(const unsigned long);
  ~Zip();
  void write () cout<< code << endl;
  void expand (const char *);
};

ZipC::ZipC()
{
  code = new char[MinZip+1];
  for (int i = 0;i<MinZip;i++)
  code[i]=InitChar;
  code[i] = '\0';
}
```

```
ZipC::ZipC (const char* zipstr)
{
  int len = (strlen(zipstr)<MaxZip) ?
  strlen(zipstr) :  MaxZip;
  code = new char [len+1];
  code[len]='\0;
}

ZipC::ZipC(const unsigned long zipnum)
{
  char buffer[BigZip+1];
  sprintf (buffer, "%d*%d",MinZip,zipnum);
  buffer[MinZip] = '\0';
  code = new char[strlen(buffer)+1];
  strcpy (code, buffer);
}
```

```
ZipC::~ZipC()
{
  delete [] code; }

void ZipC::expand (const char* suffix)
{
  char temp[BigZip+1];
  char previous[MinZip+1];
  if (strlen(code)!=MinZip ||
      strlen(suffix)!=SuffixLen)
    return;
  strcpy (previous,code);
  delete[] code;
  code=new char [BigZip+1];
  sprintf (code,"%s%c%s",
    previous,Hyphen,suffix);
}
```

- Two of the tasks of constructors are to initialize data members and dynamically allocate memory.
- C++ provides a special member function called the destructor function.
- Destructor is automatically invoked when an object is destroyed.
- Needed especially when there are dynamically allocated memory spaces.
- There are many ways to construct an object (more than one constructor), but there is only one way to destroy an object (only one destructor).

- Properties of destructor functions
- Its name is the tilde (~) character followed by the class name.
- It cannot have a return type.
- It cannot have arguments.
- It is automatically called when an object goes out of scope.

```
class ErrMessage {
  char *mes;
public:
  Errmessage (char *em) {
    mes=new char[strlen(em)];
    strcpy (mes, em);
  }
    // other member functions
};
```

There should be a destructor

```
~ErrMessage()delete [] mes;
```

A battery example

```
class Battery {
private:
  float voltage;
  float capacity;
public:
  Battery() {
    voltage=12;
    capacity=20;
  }

  battery(float v, float cp) {
    voltage=v;
    capacity=cp;
  }
```

```
  void getValues() {
    cout<<setiosflags(ios::fixed)<<setprecision(1);
    cout<<"voltage= "<<voltage<<" [V]"<<endl;
    cout<<"Capacity = "<<capacity<<" [Ah]"<<endl;
  }

  float getHours(float imp) {
    float current=voltage/imp;
    return capacity/current;
  }

  ~ Battery() {cout<<"\Battery discharged!!";}
};
```

```
int main()
{
  float imp=100;
  {
    Battery b1;
    cout<<"First battery:  "<<endl;
    b1.getValue();
    cout<<"Device can be powered "<<b1.getHours(imp);
    cout<<" hours,"<<endl;
  } //Calls destructor to destroy b1
```

```
  Battery b2(12,10);
  cout<<"\n\nSecond battery:  "<<endl;
  b2.getValues();
  cout<<"Device can be power "<<b2.getHours(imp);
  cout<<" hours."<<endl;
  return 0;
}
```